

Теория и технология программирования

Основы программирования на языках С и С++

**Лекция Объекты-контейнеры, вектора,
исключения**

Что такое контейнер?

- ❑ Контейнер предназначен для упорядоченного хранения некоторого количества однотипных данных
- ❑ Самый простой пример – стандартный массив в языке C
- ❑ Контейнер удобно описать в виде класса (а ещё лучше – в виде **шаблона** класса)
- ❑ Какие виды контейнеров вам известны?

Известные контейнеры

- ☐ Массив / вектор / список
- ☐ Упорядоченный список
- ☐ Множество
- ☐ Очередь / стек / дек (очередь с двумя концами)
- ☐ Ассоциативный массив / таблица
- ☐ Хэш-таблица
- ☐ Дерево / бинарное дерево

Очередь и стек

☐ Стек



☐ Односторонняя очередь



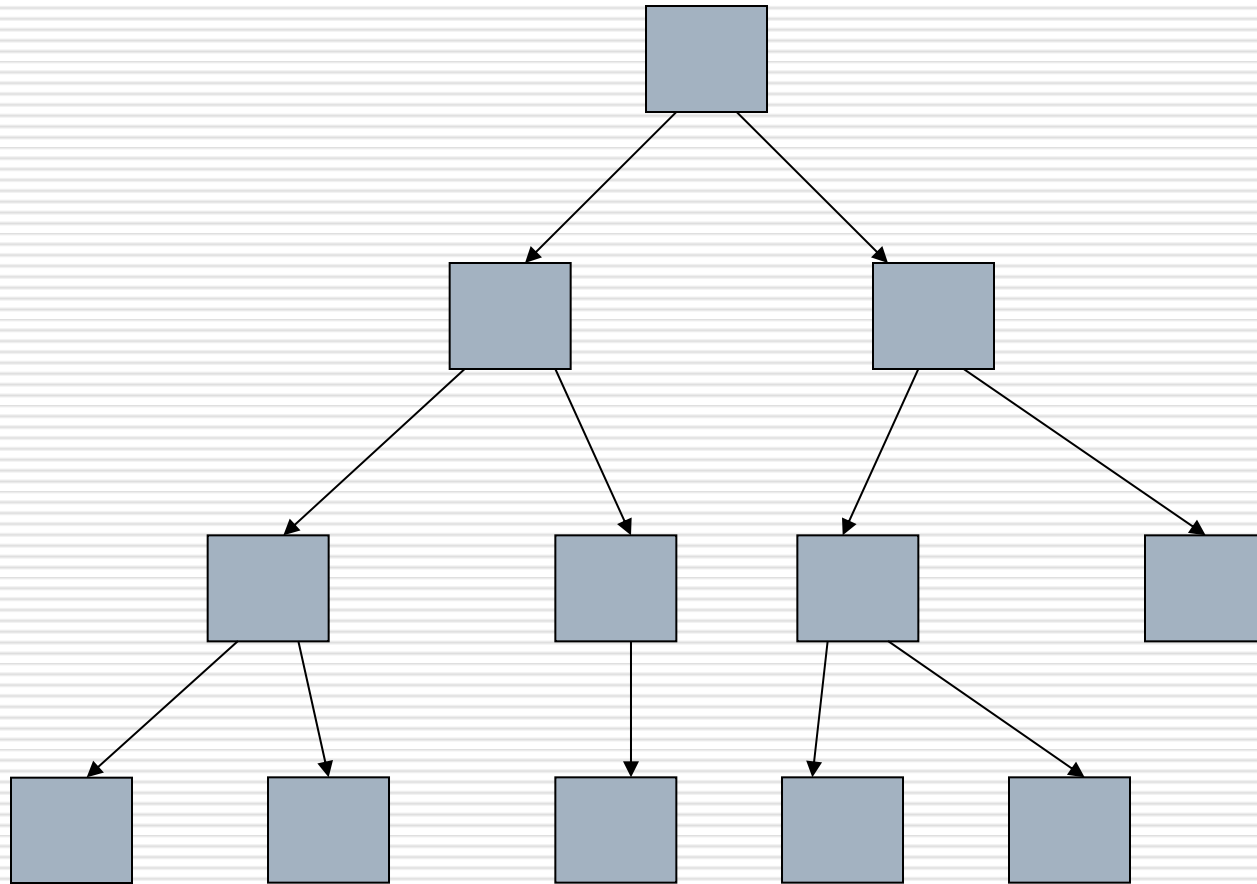
☐ Двухсторонняя очередь



☐ Реализация? Использование?

☐ Достоинства? Недостатки?

Бинарное дерево – разновидность общего дерева



Типичные операции с контейнерами

- ❑ Доступ (чтение/запись) к элементу по его номеру (индексу)
- ❑ Добавление нового элемента в конец контейнера, начало контейнера, вставка нового элемента в середину
- ❑ Удаление элемента из конца контейнера, из начала контейнера, из середины контейнера
- ❑ Поиск элемента по его значению
- ❑ Перебор элементов контейнера с выполнением какой-либо операции над каждым (итерирование)

Массив (низкоуровневый)

```
int arr[20]; // или  
int* arr = new int[size];
```

□ Достоинства

- высокая скорость доступа по индексу, $O(1)$
- высокая скорость вставки/удаления в конец, $O(1)$

□ Недостатки

- ограниченный размер, нет контроля индекса
- низкая скорость вставки/удаления в начало и середину, $O(N)$
- медленный поиск, $O(N)$

Массив (высокоуровневый), он же вектор (vector)

- Дополнительные возможности
 - изменение размера по необходимости
 - возможность контроля индекса (at)
- Достоинства
 - скорость доступа к элементам по индексу остается высокой, $O(1)$
 - скорость вставки/удаления в конец массива остается высокой, $O(1)$ (*)
- Недостатки
 - скорость вставки/удаления в начало и середину массива остается низкой, $O(N)$
 - скорость поиска остается низкой, $O(N)$
- Как бороться с недостатками?

Массив (высокоуровневый) – целые числа

```
class Array
{
    // Хранилище элементов
    int* ptr;
    // Размеры
    int size, capacity;
public:
    // Индексация
    int& operator[](int index);
    // Вставка элемента
    void insert(int elem, int index);
    // Удаление элемента
    void remove(int index);
    // ...
};
```

Свойства вектора

- ❑ Вектор имеет размер (size) – количество фактически используемых элементов
- ❑ Также вектор имеет вместимость (capacity) – фактическое число элементов в памяти (может быть больше числа используемых элементов)
- ❑ Возможность увеличить предельный размер в случае необходимости

Действия над вектором

- ❑ Чтение и запись элемента по индексу `[]` или `at`
- ❑ Вставка элемента в массив (с раздвиганием остальных) и удаление элемента из массива (со сдвиганием остальных)

Реализация вектора

- ❑ Элементы вектора необходимо хранить в динамической памяти
- ❑ Изначально создаем участок стартового размера
- ❑ Если в процессе использования этого участка оказалось мало, создаем участок двойного размера и копируем туда уже существующие элементы, а старый участок освобождаем

Необходимые данные-члены

```
class Array
{
    // Указатель на массив в динамич. памяти
    int* ptr;
    // Текущий размер
    int size;
    // Вместимость
    int capacity;
    ...
};
```

Конструктор

- ❑ В конструкторе по умолчанию можно создать массив некоторого определенного заранее размера, например, 10:

```
Array arr;
```

- ❑ Можно также определить конструктор с целым аргументом, создающий массив, размер которого задан этим аргументом:

```
Array arr(20);
```

// А эта запись нежелательна - explicit

```
Array arr=20;
```

- ❑ В такой ситуации можно использовать аргумент по умолчанию

Конструктор

// В файле array.h

```
const int DEFAULT_CAPACITY=10;
```

```
class Array
```

```
{
```

```
...
```

```
    explicit Array(int startCapacity=DEFAULT_CAPACITY);
```

```
};
```

// В файле array.cpp

```
Array::Array(int startCapacity)
```

```
{
```

```
    if (startCapacity <= 0)
```

```
        capacity = DEFAULT_CAPACITY;
```

```
    else
```

```
        capacity = startCapacity;
```

```
    ptr = new int[capacity];
```

```
    size = 0;
```

```
}
```

Деструктор

- ❑ В отличие от примера с рациональным числом, здесь мы выделяем в конструкторе память (используем **new**).
- ❑ Это означает, что выделенную память кто-то должен освободить при разрушении объекта
- ❑ Для этого определяется еще одна специальная функция – деструктор

Деструктор

- ❑ Функция, которая вызывается **автоматически** в момент разрушения объекта (по окончании его времени жизни)
- ❑ По умолчанию деструктор не делает ничего
- ❑ Название деструктора всегда состоит из символа ~, за которым следует имя типа
- ❑ У деструктора нет аргументов и возвращаемого значения

Деструктор

```
// В файле array.h
class Array
{
    ~Array();
};
// В файле array.cpp
Array::~~Array()
{
    delete[] ptr;
}
// Пример использования
int main(void)
{
    Array arr; // Здесь вызовется конструктор
    return 0; // А здесь вызовется деструктор
}
```

Конструктор копирования

- ❑ Что произойдет, если скопировать содержимое объекта Array побайтно?
- ❑ ...
- ❑ Поэтому необходим собственный конструктор копирования
- ❑ Он должен выделить участок памяти того же размера, что в массиве-аргументе, скопировать размеры и элементы

Конструктор копирования

```
// В файле array.h
class Array
{
    ...
    Array(const Array& arr);
};

// В файле array.cpp
Array::Array(const Array &arr)
{
    ptr = new int[arr.capacity];
    size = arr.size;
    capacity = arr.capacity;
    for (int i=0; i<size; i++)
        ptr[i]=arr.ptr[i];
}
```

Оператор присваивания

- ❑ По той же причине, необходим собственный оператор присваивания
- ❑ В отличие от конструктора копирования, он вначале должен удалить уже выделенный участок памяти
- ❑ А уже потом создать новый

Оператор присваивания

```
// В файле array.h
class Array
{
    ...
    Array& operator =(const Array& arr);
};
```

Оператор присваивания

// В файле array.cpp

```
Array& Array::operator =(const Array& arr)
{
    if (this==&arr)
        return *this;
    if (capacity != arr.capacity)
    {
        delete[] ptr;
        ptr = new int[arr.capacity];
        capacity = arr.capacity;
    }
    size = arr.size;
    for (int i=0; i<size; i++)
        ptr[i] = arr.ptr[i];
    return *this;
}
```

Чтение и запись элементов массива

- В некоторых случаях, для этой цели определяется пара функций `get` и `set`, например:

```
class Array
{
    ...
    // Получение элемента с заданным индексом
    int get(int index) const;
    // Установка значения элемента с заданным
    индексом
    void set(int index, int elem);
};
```


Чтение и запись элементов массива

- ❑ В C++, однако, существует более красивое решение – перегрузка оператора индексации []
- ❑ Универсальная версия (пригодная как для чтения, так и для записи) выглядит так:

```
class Array
{
    ...
    int& operator [] (int index);
};
```

- ❑ При выполнении команд вида

```
Array arr;
```

```
...
```

```
arr[2]=4;
```

- ❑ Произойдет вызов функции `operator []` с аргументом 2, а затем запись по ссылке числа 4, эквивалентно `(arr.operator [] (2))=4;`

Чтение и запись элементов массива

- ❑ Возможна также версия, пригодная только для чтения – результат в этом случае не ссылка на элемент массива, а его значение:

```
class Array
{
    ...
    int operator [] (int index) const;
};
```

- ❑ В отличие от предыдущего варианта, такая функция не приводит к изменению данных массива.

Реализация оператора индексации

```
// Файл array.cpp
int& Array::operator [](int index)
{
    // Индекс должен быть в пределах 0...size-1
    // При таком подходе нельзя обратиться к элементу,
    // который еще не был создан операцией вставки
    if (index >= size || index < 0)
        ... // ???
    else
        return ptr[index];
}
```

- Встает вопрос - а что делать, если значение индекса оказалось некорректно?

Варианты обработки ошибочных ситуаций

- Вернуть заведомо невозможный результат?
 - Поскольку в нашем случае результат – ссылка, то это невыполнимо. Ссылка всегда будет соответствовать какой-то конкретной целой переменной.
 - Даже если результат – значение, то это все равно невыполнимо. Любое целое значение может храниться в массиве.

Варианты обработки ошибочных ситуаций

- Вывести сообщение об ошибке и закончить программу?
 - Если наш класс является частью большой программы, то это нежелательно. Подобные маленькие фрагменты не должны решать, завершать программу или нет. Кроме этого, не у каждой программы есть куда выводить сообщение об ошибке.

Варианты обработки ошибочных ситуаций

- ❑ Ввести в объекте «массив» признак ошибочного завершения операции и функцию для чтения признака:

```
class Array
{
    ...
    bool isFail;
public:
    ...
    bool fail();
};
```

Варианты обработки ошибочных ситуаций

- ❑ Ввести в объекте «массив» признак ошибочного завершения операции и функцию для чтения признака
- ❑ Тогда при возникновении ошибки `isFail` будет устанавливаться в `true`
- ❑ При использовании данного класса, вызов функции `fail()` показывает, была ошибка или нет
- ❑ Подобное решение применено, например, для потоков. Тем не менее оно не слишком удачное, так как легко забыть вызвать функцию `fail()`

Варианты обработки ошибочных ситуаций

- ❑ Не обрабатывать ошибку в принципе (то есть, не проверять значение индекса, полагаясь на то, что оно правильное)
- Подобное решение применяется в ситуациях, когда хотят повысить производительность. Вообще говоря, оно нежелательно, так как ошибка с выходом индекса за границу массива встречается очень часто. Промежуточный вариант – `assert`

Обработка ошибочных ситуаций в C++

- ❑ Используется механизм «исключений»
- ❑ Исключение с точки зрения C++ - это переменная, содержащая информацию о произошедшей ошибке; подобная переменная может быть любого типа (как встроенного, так и пользовательского)
- ❑ Чаще всего каждой ошибке ставится в соответствие исключение своего типа

Пример использования исключений

```
// Класс без данных и методов -  
// простейший вариант исключения  
class Exception {};  
// Функция f вызывает функцию g,  
// в которой может произойти исключение  
void f(int k)  
{  
    try  
    {  
        int res=g(k);  
    } catch (Exception& e)  
    {  
        cout<<"Произошла ошибка!!"<<endl;  
    }  
}
```

Пример использования исключений

```
// Класс без данных и методов –  
// простейший вариант исключения  
class Exception {};  
// ...  
// Функция g, в которой может произойти исключение  
int g(int n)  
{  
    if (...) // Если произошла ошибка  
        throw Exception();  
    return n+1;  
}
```

Как работают исключения

- Если в функции *g* генерируется исключение (**throw** *Exception*):
 - если исключение произошло внутри конструкции **try/catch** (*Exception&*), то остаток блока **try** пропускается, выполняется блок **catch**
 - если исключение произошло вне конструкции **try/catch** (*Exception&*), то выполнение функции *g* прерывается, результат не формируется, исключение передается в функцию на уровень выше (*f*)
- Далее проверяется, был ли вызов функции *g* внутри конструкции **try/catch** (*Exception&*) уже в функции *f*; если нет, то еще на уровень выше; и так далее, вплоть до функции *main*.
- В приведенном примере выполнение функции *g* прервется, в функции *f* остаток блока **try** будет пропущен, начнет выполняться блок **catch** в функции *f*

В чем достоинства исключений?

- ❑ Мы не должны каждый раз при вызове функции проверять, произошла ошибка или нет
- ❑ Мы можем использовать результат функции по своему усмотрению (а не передавать через него признак ошибки)
- ❑ Мы можем устранить ошибку там, где нам удобно. Например, в крупных проектах на верхнем уровне часто происходит обработка вообще **всех** ошибок, которые не были обработаны на нижних уровнях – обычно при этом выполняемое действие прерывается и приложение ожидает команды пользователя для выполнения следующего действия

Как применить исключения в нашей задаче?

```
// Файл array.h
class ArrayException {};
class Array
{
    ...
    int& operator [] (int index);
};

// Файл array.cpp
int& Array::operator [] (int index)
{
    if (index >= size || index < 0)
        throw ArrayException();
    else
        return ptr[index];
}
```

Функция вставки элемента

- ❑ Должна вставить заданный элемент в заданную позицию массива.
- ❑ Бывший на этой позиции элемент и следующие за ним должны быть сдвинуты на одну ячейку
- ❑ Если необходимо, предельный размер массива должен быть увеличен (должен быть выделен участок динамической памяти большего размера)

// Файл array.h

```
class Array  
{
```

...

```
void insert(int index, int elem);
```

// Вставка элемента в конец массива

```
void insert(int elem);
```

```
};
```

Реализация

- Есть три варианта индекса:
 - некорректное значение индекса (<0 или $>\text{size}$)
 - значение индекса $0 \leq \text{index} \leq \text{size}-1$ – тогда надо сдвинуть на 1 все элементы, номер которых $\geq \text{index}$, и на место index вставить новый; размер массива увеличивается на 1
 - значение index равно size – туда вставляется новый элемент, размер массива увеличивается на 1
- Если size превысило capacity (предельный размер массива), то необходимо увеличить capacity и заново выделить память

Текст функции

```
void Array::insert(int elem, int index)
{
    if (index < 0 || index > size)
        throw ArrayException();
    if (size==capacity)
        // Закрытая функция, увеличивающая предельный размер
        increaseCapacity(size+1);
    // Если index==size, этот цикл не выполнится ни разу
    for (int j=size-1; j>=index; j--)
        ptr[j+1]=ptr[j];
    size++;
    ptr[index]=elem;
}

void Array::insert(int elem)
{
    insert(elem, size);
}
```

Увеличение предельного размера

- ❑ Аргумент функции – требуемый предельный размер
- ❑ Удобно увеличивать предельный размер скачками – если каждый раз увеличивать его на 1, производительность упадет
- ❑ Пусть каждый раз, когда требуется увеличение предельного размера, он удваивается

Увеличение предельного размера

// Файл array.h

class Array

```
{  
    void increaseCapacity(int newCapacity);  
};
```

// Файл array.cpp

```
void Array::increaseCapacity(int newCapacity) {  
    capacity = newCapacity < capacity*2 ?  
        capacity*2 : newCapacity;  
    int* newPtr = new int[capacity];  
    for (int i=0; i<size; i++)  
        newPtr[i]=ptr[i];  
    delete[] ptr;  
    ptr = newPtr;  
}
```

Функция удаления элемента

- ❑ Должна удалить элемент из заданной позиции массива.
- ❑ Следующие за этой позицией элементы должны быть сдвинуты на одну ячейку назад
- ❑ Предельный размер массива при этом не меняется

```
// Файл array.h
class Array
{
    ...
    // Удаление элемента
    void remove(int index);
};
```

Реализация

- ❑ Значения индекса < 0 и $\geq \text{size}$ некорректны
- ❑ Элементы с номерами $\text{index}+1 \dots \text{size}-1$ должны быть сдвинуты на 1 позицию назад
- ❑ Размер должен быть уменьшен на 1

Текст функции

```
void Array::remove(int index)
{
    if (index < 0 || index >= size)
        throw ArrayException();
    for (int j=index; j<size-1; j++)
        ptr[j]=ptr[j+1];
    ptr[size-1]=0;
    size--;
}
```

Другие функции

```
// Получение размера  
// Вывод массива в поток  
// Файл array.h  
class Array  
{  
    int getSize() const;  
    friend ostream& operator <<(ostream& out,  
        const Array& arr);  
};
```

Реализация

```
int Array::getSize() const
{
    return size;
}

ostream& operator <<(ostream& out,
    const Array& arr)
{
    out<<"Total size: "<<arr.size<<endl;
    for (int i=0; i<arr.size; i++)
        out<<arr.ptr[i]<<endl;
    return out;
}
```


Тестирование

```
int main(void)
{
    setlocale(LC_ALL, "Russian");
    Array arr(4);
    for (int i=0; i<4; i++)
        arr.insert(i+1);
    cout<<arr<<endl;
    for (int i=0; i<8; i+=2)
        arr.insert(10+i,i);
    cout<<arr<<endl;
    for (int i=1; i<8; i+=2)
        arr[i]=20+i;
    cout<<arr<<endl;
    for (int i=6; i>=0; i-=3)
        arr.remove(i);
    cout<<arr<<endl;
}
```

Шаблон <vector>

- ❑ Внутренняя организация – на основе массива
- ❑ Размер массива увеличивается по мере необходимости
- ❑ Похож на наш класс Array

Основные свойства вектора

- `size()` – *размер* – количество элементов, имеющих в векторе на данный момент
 - может быть изменено вызовом `resize(int)`
 - оператор индексации позволяет обратиться к элементам `0...size()-1`
- `capacity()` – *вместимость* – количество элементов, под которые на данный момент выделена память
 - всегда верно `capacity() >= size()`
 - может быть изменено вызовом `reserve(int)`
- `max_size()` – максимально возможное количество элементов в принципе

Основные возможности вектора

☐ Конструкторы

- по умолчанию (пустой вектор)
- заданного размера
- заданного размера и наполнения
- копирования

☐ Оператор присваивания

☐ Операторы сравнения

☐ Обращение по индексу `[i]` или `at(i)`

- диапазон значений `0...size()-1`
- размер не может меняться
- в `at` при неверном индексе происходит ошибка при выполнении программы

Основные возможности вектора

- ❑ Очистка (clear)
- ❑ Проверка на пустоту (empty)
- ❑ Добавление последнего элемента (push_back)
 - размер увеличивается на 1, вместимость увеличивается при необходимости
- ❑ Удаление последнего элемента (pop_back)
 - размер уменьшается на 1

Результат

- ❑ Разработан и оттестирован тип «массив целых чисел изменяемого размера»
- ❑ Поддерживаются операции индексации, вставки, удаления, вывода, копирования
- ❑ Что делать, если понадобится, например, массив вещественных чисел?
 - используя аппарат **шаблонов (template)**, можно реализовать массив для хранения любых однотипных данных
 - шаблоны мы рассмотрим на следующей лекции